

Архитектура процессора 8086



Intel 8086 – первый 16-битный микропроцессор

Дата анонса: 8 июня 1978 года

- Разрядность регистров: 16 бит
- Разрядность шины данных: 16 бит
- Разрядность шины адреса: 20 бит

Объём адресуемой памяти: 1 Мбайт

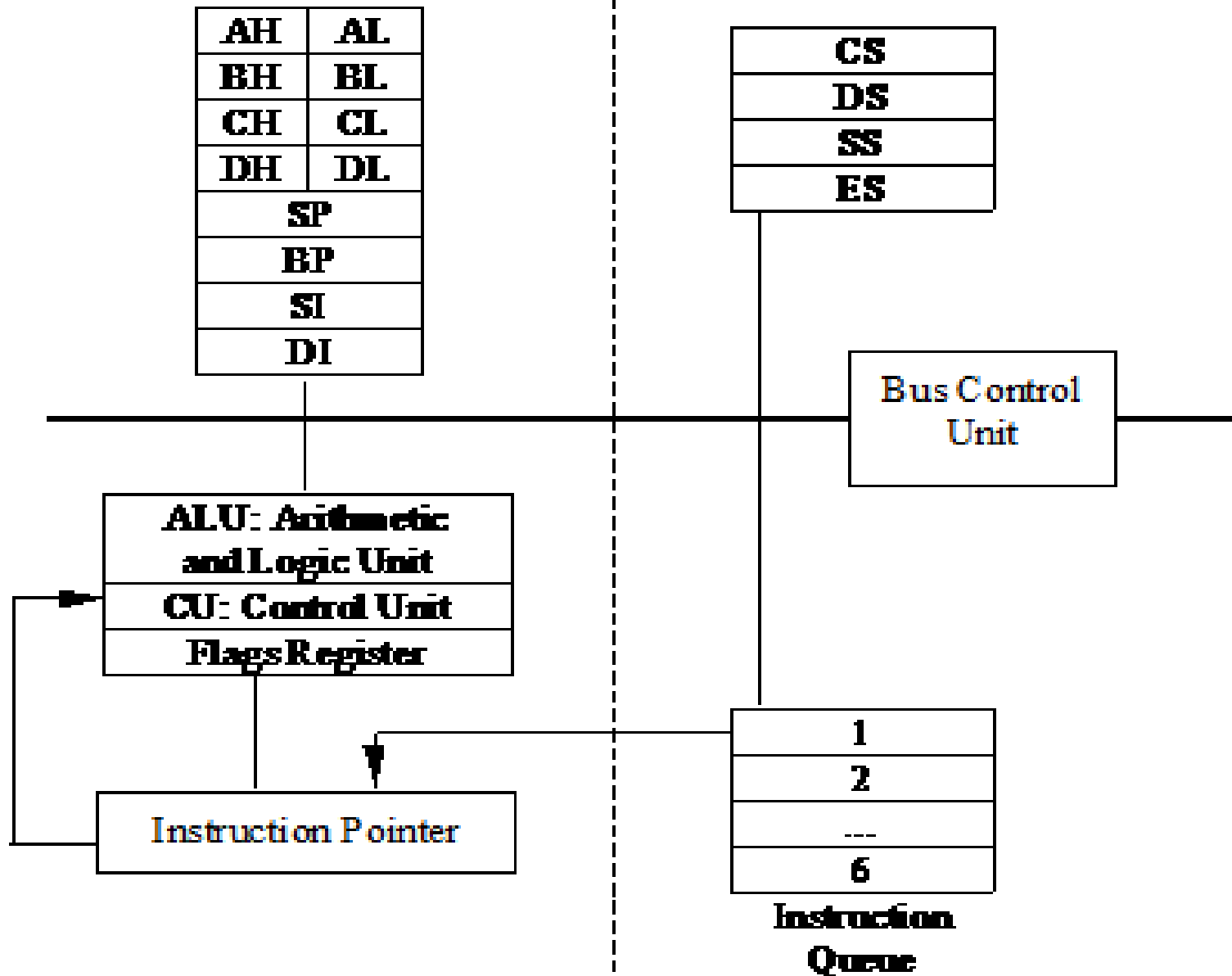
Адресное пространство I/O: 64 Кбайт

Поддерживаемые технологии: 98 команд

Объём очереди команд: 6 байт

EU: Execution Unit

BIU: Bus Interface Unit



Структура центрального процессора Intel 8086

Два логических блока:

- блок исполнения (EU:Execution Unit);
- блок интерфейса шин (BIU:Bus Interface Unit).

Интерфейс (interface) - это совокупность средств, обеспечивающих сопряжение устройств и программных модулей как на физическом, так и на логическом уровнях.

Блок исполнения EU

В состав EU входят:

- арифметическо-логическое устройство ALU,
- устройство управления CU (Control Unit)
- десять регистров.

Устройства блока EU обеспечивают обработку команд, выполнение арифметических и логических операций.

Блок интерфейса шин BIU

Три части блока BIU - устройство управления шинами, блок очереди команд и регистры сегментов – предназначены для выполнения следующих функций:

- управление обменом данными с EU, памятью и внешними устройствами ввода/вывода;
- адресация памяти;
- выборка команд (осуществляется с помощью блока очереди команд Queue, который позволяет выбирать команды с упреждением).

Процессор 8086 состоит из:

- 8 регистров общего назначения,
- 4 сегментных регистров,
- регистра адреса команд (счетчика команд)
- регистра флагов.

Процессор выставляет на шину адреса адрес выбираемых из памяти команд (или данных), которые поступают в шестибайтный буфер (очередь команд), а затем исполняются.

Адресная шина

20 проводников, в каждом из которых может либо протекать напряжение заданного уровня (сигнал 1), либо отсутствовать (сигнал 0).

Таким образом, микропроцессор оперирует с двоичной системой счисления (двоичной системой представления данных):

- Символьная информация кодируется в соответствии с кодом ASCII (Американский стандартный код для обмена информацией).
- Числовые данные кодируются в соответствии с двоичной арифметикой.
- Отрицательные числа представляются в дополнительном коде.

Регистры процессора

Регистры – это 16-разрядные ячейки памяти расположенные в процессоре. Их достоинство заключается в гораздо большем быстродействии, чем у оперативной памяти. Недостаток – их очень мало – 14 регистров.

Регистры делятся на пять категорий:

- Регистры общего назначения (ax, bx, cx, dx)
- Регистровые указатели и индексные регистры (sp, bp, si, di) – адресные регистры
- Сегментные регистры (cs, ds, ss, es)
- Регистр командного указателя (ip)
- Регистр флагов

	16 bit		
	8 bit	8 bit	
AX:	AH	AL	Сумматор
BX:	BH	BL	Базовый регистр
CX:	CH	CL	Счетчик
DX:	DH	DL	Регистр данных

SP	Стековый указатель
BP	Базовый указатель
SI	Индекс источника
DI	Индекс назначения

CS	Регистр сегмента кода
DS	Регистр сегмента данных
SS	Регистр сегмента стека
ES	Дополнительный сегментный регистр

IP	Командный указатель
-----------	----------------------------

##### of df if tf sf zf # af # pf # cf	Регистр флагов
---	-----------------------

1. Регистры общего назначения

Это регистры данных, каждый из которых помимо хранения операндов и результатов операций имеет еще и свое специфическое назначение:

- **регистр AX** (*accumulator*) — автоматически применяется в операциях умножения, деления, и при работе с портами ввода-вывода (команды `in`, `out`);
- **регистр BX** (*base*) — регистр базы может содержать адреса элементов в оперативной памяти, которые по умолчанию представляют собой смещение в сегменте данных;
- **регистр CX** (*count*) — счетчик циклов, определяет количество повторов некоторой операции, например в циклах, в строковых командах и т.п.;
- **регистр DX** (*data*) — определение адреса ввода/вывода, так же может содержать данные, передаваемые для обработки в подпрограммы или применяется в операциях умножения и деления в качестве расширителя аккумулятора.

Эти определения, не являются полными и обязательными, и большую часть времени программист сам решает, как использовать регистры общего назначения.

Однако в ряде случаев некоторые команды процессора 8086 требуют строго определенных регистров.

Регистры AX, BX, CX, и DX позволяют независимо обращаться к их старшей и младшей половине. Соответствующие подрегистры являются восьмиразрядными и имеют имена AH, AL, BH, BL, CH, CL, DH, DL.

2. Адресные регистры

Существуют четыре 16-битовых регистра, которые могут участвовать в адресации операндов.

Регистровые указатели SP и BP обеспечивают доступ к данным в сегменте стека.

- **регистр SP** (*Stack Pointer*) – указатель стека – это 16-битовый регистр, который используется для записи данных в стек и чтения их из стека. Фактически он содержит смещение в сегменте стека, которое определяет нужное слово памяти. Значение этого регистра автоматически изменяется командами для работы со стеком типа push, pop, pushf, popf, call, ret;
- **регистр BP** (*Base Pointer*) – обычно адресует переменные, хранимые в стеке, облегчает доступ к параметрам (данным и адресам), переданным через стек.

Индексные регистры SI и DI могут применяться для расширенной адресации, для использования в операциях сложения и вычитания, а так же для операций над байтовыми строками (в языке ассемблера байтовая строка представляет собой просто ряд последовательных байт).

- **регистр SI** (*Source Index*) – является индексом источника и применяется для некоторых операций над строками (обычно связан с регистром DS);
- **регистр DI** (*Destination Index*) – является индексом назначения и применяется так же для строковых операций. При выполнении операций со строками в этом регистре содержится смещение строки приемника в сегменте дополнительных данных (обычно связан с регистром ES).

Сегменты

$$1 \text{ Мб} = 1024 \text{ Кб} = 1024 * 1024 \text{ байт} = 1048576 \text{ байт} = 2^{20}$$

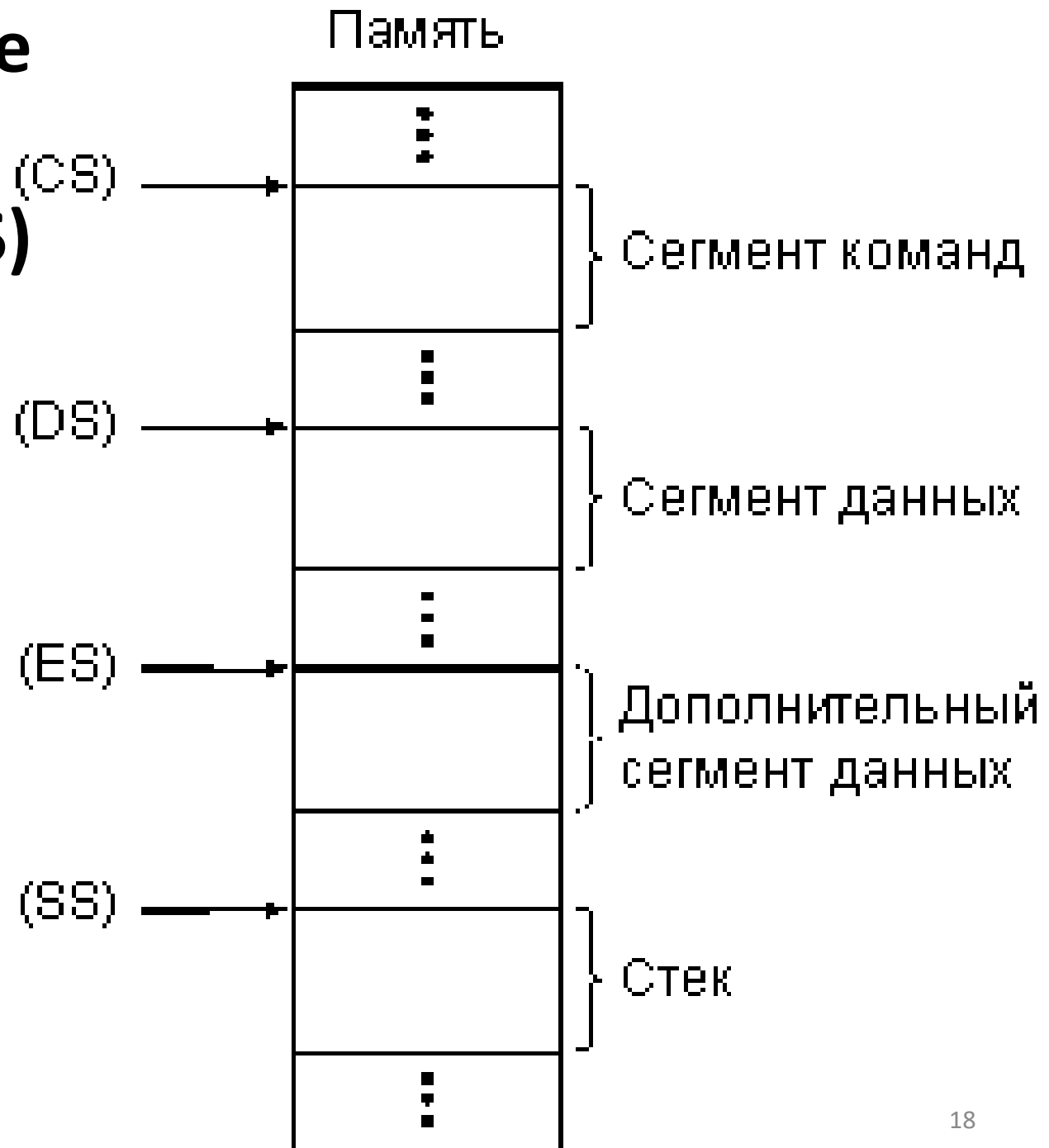
Сегментом называется область, которая начинается на границе параграфа, т.е. по любому адресу, который делится на 16 без остатка. Хотя сегмент может располагаться в любом месте памяти и иметь размер до 64 Кбайт, он требует столько памяти, сколько необходимо для выполнения программы.

Внутри программы все адреса памяти относительно к началу сегмента. Такие адреса называются смещением от начала сегмента. Двухбайтовое смещение (16-бит) может быть в пределах от 0000_{16} до $FFFF_{16}$ или от 0 до 65535. Для обращения к любому адресу в программе, компьютер складывает адрес в регистре сегмента и смещение.

Пример адресации: допустим, что регистр сегмента данных содержит 045F и некоторая команда обращается к ячейке памяти внутри сегмента данных со смещением 0032. Несмотря на то, что регистр сегмента данных содержит 045F, он указывает на адрес 045F0, т.е. на границе параграфа. Действительный адрес памяти поэтому будет следующий:

Адрес в DS:	045F0
Смещение:	0032
Реальный адрес:	04622

3. Сегментные регистры (CS, DS, SS, ES)



Определяют в памяти начала четырех 64 Кбайтовых сегментов, которые называются *текущими сегментами*.

Сегментные регистры строго специализированы. С помощью них нет возможности выполнять математические вычисления или хранить в них результаты других операций.

Сегменты могут в любом порядке храниться в произвольных местах памяти.

- **регистр CS** (*Code Segment*) – содержит начальный адрес сегмента кода (начало машинного кода программы). Этот адрес плюс значение смещения в командном указателе (IP) определяет адрес команды, которая должна быть выбрана для выполнения.
- Физический адрес команды в памяти для выполняемой программы определяет пара регистров CS и IP и это записывается в виде CS:IP. Аналогичная форма записи используется для указания физического адреса и в других сегментах.
- Это единственный сегментный регистр, который нельзя загрузить непосредственно. Косвенно загрузить в CS новое значение могут только команды вида jxx, call, int, ret, iret.

- **регистр DS** (*Data Segment*) – содержит начальный адрес сегмента данных (переменных, строк и т.п. данных, которыми оперирует программа). По умолчанию смещения в сегменте данных задаются в регистрах BX, SI и DI.
- **регистр SS** (*Stack Segment*) – регистр сегмента стека определяет стартовый адрес сегмента, в который помещается стек для программы. По умолчанию смещения в сегменте стека задаются в регистрах SP и BP.

- **регистр ES** (*Extra Segment*) – является вспомогательным регистром, используется при некоторых операциях над строками. В большинстве программ в ES и DS содержатся одинаковые адреса, но он может упрощать некоторые операции связанные с этими регистрами. Например, в случае строковых команд, DS определяет сегмент для строки-источника, а ES – сегмент для строки-приемника.

4. Регистр указателя команд (IP)

- **Регистр командного указателя IP** (*Instruction Pointer*) содержит смещение на команду, которая должна быть выполнена. Обычно при программировании этот регистр не используют. Но очень удобно наблюдать за изменениями его значений при отладке программы в различных отладчиках.

- Обычно команды выполняются в естественном порядке, то есть так, как они располагаются в программе. Нарушают естественную последовательность только команды переходов (они начинаются с буквы `j` и обычно обозначаются как `jxx`), команды вызова подпрограмм (`call`), обработчиков прерываний (`int`) и возврата (`ret`, `iret`).
- Непосредственно содержимое `IP` нельзя изменить или прочитать. Косвенно загрузить в регистр `IP` новое значение могут только команды вида `jxx`, `call`, `int`, `ret`, `iret`.

5. Регистр флагов (состояния)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
				OF	DF	IF	TF	SF	ZF		AF		PF		CF

Это регистр состояния процессора. Состоит из 16 бит, из них используются только 9.

Биты регистра состояния устанавливаются или очищаются в зависимости от результата исполнения предыдущей команды и используются некоторыми командами процессора.

Биты регистра состояния могут также устанавливаться и очищаться специальными командами процессора (pushf и popf).

- **CF** (*Carry Flag*) — флаг переноса при арифметических операциях. Содержит перенос из старшего бита после арифметических операций, а так же последний бит при сдвигах или циклических сдвигах. Может быть установлен в ноль командой `clic` и в единицу командой `stc`.
- **PF** (*Parity Flag*) — флаг четности результата, показывает четность младших восьмибитовых данных (1 — четное, 0 — нечетное).
- **AF** (*Auxiliary Flag*) — флаг дополнительного переноса. Используется для выполнения операций десятичной арифметики, когда данные хранятся в двоично-десятичном коде. Этот бит фиксирует перенос из младшей тетрады в старшую при сложении либо заем при вычитании.

- **ZF** (*Zero Flag*) — флаг нулевого результата. Показывает результат арифметических операций и операций сравнения (0 – ненулевой, 1 – нулевой результат).
- **SF** (*Sign Flag*) — флаг знака. Совпадает со старшим битом результата (0 – плюс, 1 – минус).
- **TF** (*Trap Flag*) — флаг пошагового режима (используется при отладке). Этот бит устанавливается программистом. Если T=1, то по завершению каждой команды возникает прерывание работ микропроцессора.

- **IF** (*Interrupt-enable Flag*) — флаг разрешения аппаратных прерываний.
 - При $I=1$ микропроцессор реагирует на внешние аппаратные прерывания по входу INTR.
 - При $I=0$ прерывания по входу INTR запрещаются или маскируются.

Значение признака I не влияет на восприятие внешних немаскируемых прерываний (вход NMI микропроцессора) и на выполнение внутренних или программных прерываний по команде `int`.

Этот признак может быть установлен в ноль командой `cli` и в единицу командой `sti`.

- **DF** (*Direction Flag*) — флаг направления при строковых операциях.
 - Если $D=0$, команды, работающие с блоками, увеличивают содержимое индексных регистров для выбора следующего элемента.
 - Если $D=1$, то команды уменьшают содержимое этих регистров.

Этот признак может быть установлен в ноль командой `cld` и в единицу командой `std`.
- **OF** (*Overflow Flag*) — флаг переполнения.

Устанавливается в единицу в случае переполнения при выполнении операций сложения и вычитания со знаковыми числами, тем самым указывает на потерю старшего бита результата.